

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP DOANH NGHIỆP

CHATBOT HỖ TRỢ SINH VIÊN VÀ
TƯ VẤN TUYỂN SINH

Khoa/Viện:

Trí tuệ nhân tạo

Giảng viên hướng dẫn:

PGS.TS.Nguyễn Việt Hà

ThS. Nguyễn Thị Thuỳ Linh

Sinh viên thực hiện:

Nguyễn Hải Nam - 22022600

Lê Anh Tiến - 22022528

Hà Xuân Huy - 23020375

Hà Nội - 2025



Ý kiến đánh giá

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số:

Điểm chữ:

Hà Nội, ngày ... tháng ... năm 2025

Giảng viên đánh giá

Lời Cảm Ơn

Tôi xin gửi lời cảm ơn đến giảng viên hướng dẫn thầy PGS.TS.Nguyễn Việt Hà và cô TS. Nguyễn Thị Thuỳ Linh đã trực tiếp hướng dẫn, góp ý và chia sẻ kinh nghiệm quý báu để nhóm hoàn thành tốt dự án trong quá trình học môn dự án.

Tôi cũng xin chân thành cảm ơn viện Trí tuệ nhân tạo và tất cả những người đã hỗ trợ tôi trong quá trình thực tập và thực hiện dự án đã tạo điều kiện thuận lợi để tôi hoàn thành nhiệm vụ học tập và dự án của mình.

Hà Nội, ngày 20 tháng 12 năm 2025

MỤC LỤC

Tóm tắt	iv
Danh mục hình ảnh	v
Danh mục bảng	vi
1 GIỚI THIỆU	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Xác định bài toán và các vấn đề tồn tại	1
1.3 Mục tiêu và Phạm vi nghiên cứu	2
1.4 Ý nghĩa khoa học và thực tiễn của đề tài	2
1.5 Bố cục báo cáo	3
2 CƠ SỞ LÝ THUYẾT	4
2.1 Tổng quan về hệ thống Chatbot trong giáo dục	4
2.2 Mô hình ngôn ngữ và kỹ thuật huấn luyện	5
2.2.1 Mô hình nền tảng Qwen3-4B	5
2.2.2 Kỹ thuật LORA	5
2.2.3 Kỹ thuật RAFT	7
2.2.4 Kỹ thuật DPO	8
2.2.5 Tối ưu hóa suy luận với vLLM	9
2.3 Kỹ thuật tìm kiếm và tăng cường tri thức (RAG & Search)	10
2.3.1 Tổng quan kiến trúc RAG	10
2.3.2 Biểu diễn vector và tìm kiếm tương đồng	10
2.3.3 Xếp hạng lại ngữ nghĩa bằng Cross-Encoder	11
2.3.4 Tìm kiếm Web và định tuyến truy vấn	11
3 GIẢI PHÁP ĐỀ XUẤT VÀ TRIỂN KHAI	12
3.1 Tổng quan kiến trúc hệ thống Chatbot	12
3.1.1 Các thành phần chính	12
3.1.2 Luồng dữ liệu tổng quát	13
3.2 Xử lý dữ liệu và huấn luyện mô hình	14
3.2.1 Xử lý dữ liệu	14
3.2.2 Quy trình sinh câu trả lời chuẩn (Ground Truth Generation)	15

3.2.3	Quy trình Tích hợp RAFT (Data Engineering Pipeline)	15
3.2.4	Chiến lược Giai đoạn 1: Tinh chỉnh LoRA cơ bản	17
3.2.5	Chiến lược Giai đoạn 2: Tinh chỉnh LoRA + RAFT	18
3.3	Hệ thống tìm kiếm cục bộ (Local Search)	20
3.3.1	Kiến trúc và pipeline Local Search	20
3.3.2	Phối hợp Local Search với Web Search	21
3.4	Tìm kiếm thông tin từ Web (Web Search)	21
3.4.1	Tổng quan Kiến trúc Web Search	22
3.4.2	Phân tích và Sinh truy vấn (Query Generation)	22
3.4.3	Truy xuất và Xếp hạng trang (Search & Page Reranking)	23
3.4.4	Thu thập và Tiền xử lý dữ liệu (Crawling & Parsing)	23
3.4.5	Phân đoạn, Khử trùng lặp và Lọc ngữ nghĩa (Chunking, Deduplication & Semantic Filtering)	23
3.4.6	Hợp nhất ngữ cảnh và Tạo câu trả lời (Final RAG)	24
4	KẾT QUẢ VÀ ĐÁNH GIÁ	25
4.1	Phương pháp Đánh giá	25
4.2	Kết quả Đánh giá Huấn luyện	25
4.2.1	Kết quả Huấn luyện Định lượng	25
4.3	Kết quả Đánh giá hệ thống	26
5	KẾT LUẬN	28
5.1	Tổng kết Hệ thống	28
5.2	Tính năng Chính	28
5.3	Hướng Phát triển Tương lai	28

Tóm tắt

Báo cáo này trình bày kết quả nghiên cứu và phát triển phiên bản nâng cấp của hệ thống Chatbot Tư vấn Tuyển sinh, nhằm giải quyết bài toán tra cứu thông tin phân tán và biến động liên tục trong bối cảnh kỳ thi tốt nghiệp THPT năm 2025. Dựa trên nền tảng nghiên cứu trước đó, nhóm tập trung vào bốn cải tiến kỹ thuật cốt lõi:

- (i) **Tối ưu hóa mô hình ngôn ngữ:** Tinh chỉnh mô hình Qwen3-4B sử dụng kỹ thuật LoRA + trên tập dữ liệu mở rộng, giúp tăng cường khả năng thấu hiểu ngữ cảnh chuyên ngành và tuân thủ các quy chế tuyển sinh.
- (ii) **Nâng cấp kiến trúc tìm kiếm:** Xây dựng pipeline tìm kiếm lai (Hybrid Search) tích hợp bộ định tuyến (Router) thông minh, cho phép tự động phân loại và điều hướng truy vấn giữa cơ sở dữ liệu Vector cục bộ và công cụ tìm kiếm Web thời gian thực (kết hợp Brave và Google Search API). Trên nền tảng đó, phát triển pipeline tìm kiếm Web chuyên sâu (Advanced Web Search) với kiến trúc xử lý song song (Parallel Processing), kết hợp cơ chế lọc đa tầng sử dụng mô hình Cross-Encoder (BGE-M3) nhằm đánh giá mức độ liên quan ngữ nghĩa và loại bỏ hiệu quả các nguồn tin nhiễu từ Internet trước khi đưa vào mô hình sinh.
- (iii) **Chuẩn hóa dữ liệu:** Tái cấu trúc cơ sở dữ liệu Vector bao phủ hơn 200 trường đại học với cơ chế kiểm tra tính mới (freshness check), đảm bảo thông tin luôn cập nhật.
- (iv) **Hoàn thiện hệ thống:** Tối ưu hóa suy luận với thư viện vLLM trên hạ tầng Kaggle/Local và cải thiện trải nghiệm người dùng thông qua các tính năng trích dẫn nguồn minh bạch và lưu trữ hội thoại.

Kết quả thực nghiệm cho thấy hệ thống mới đạt độ chính xác cao hơn, giảm thiểu hiện tượng ảo giác và cải thiện đáng kể tốc độ phản hồi so với phiên bản tiền nhiệm.

Danh mục hình ảnh

3.1	Kiến trúc hệ thống Chatbot	12
3.2	Kiến trúc hệ thống WebSearch	22
4.1	Human Evaluation	26
4.2	LLM-as-a-Judge	27

Danh mục bảng

3.1	Cấu hình huấn luyện cho giai đoạn LoRA cơ bản	18
3.2	Cấu hình LoRA và huấn luyện trong giai đoạn LoRA + RAFT	19
3.3	So sánh định dạng dữ liệu RAFT cho mẫu có context và không có context	20
4.1	So sánh metrics huấn luyện giữa LoRA cơ bản và LoRA + RAFT	26

Chương 1

GIỚI THIỆU

1.1 Bối cảnh và lý do chọn đề tài

Kỳ thi tốt nghiệp THPT năm 2025 đánh dấu một bước ngoặt quan trọng của nền giáo dục Việt Nam khi là năm đầu tiên học sinh hoàn thành Chương trình Giáo dục phổ thông 2018 và tham gia kỳ thi với cấu trúc mới. Theo số liệu từ Bộ Giáo dục và Đào tạo, số lượng thí sinh đăng ký dự thi đạt mức kỷ lục 1,16 triệu, tăng đáng kể so với 1,07 triệu của năm 2024. Những thay đổi về cấu trúc đề thi, sự xuất hiện của các môn thi mới cùng với sự đa dạng và phức tạp của các phương thức xét tuyển đại học như đánh giá năng lực hay xét tuyển kết hợp đã tạo ra một lượng thông tin lớn, phân tán và khó tiếp cận đối với người học.

Bên cạnh công tác tư vấn tuyển sinh, sinh viên đang theo học tại các trường đại học cũng thường xuyên gặp khó khăn trong việc tra cứu thông tin học vụ, quy chế đào tạo, học phí, chương trình học và các thông báo học thuật. Thực tế cho thấy, các kênh tư vấn truyền thống và bộ phận hỗ trợ hành chính khó có thể đáp ứng nhu cầu tra cứu liên tục, đặc biệt trong các giai đoạn cao điểm như tuyển sinh, đăng ký học phần hoặc công bố kết quả học tập. Xuất phát từ yêu cầu thực tiễn đó, nhóm nghiên cứu lựa chọn đề tài xây dựng hệ thống *Chatbot hỗ trợ sinh viên và tư vấn tuyển sinh*. Đề tài ứng dụng công nghệ Trí tuệ nhân tạo nhằm cung cấp một công cụ tra cứu tự động, chính xác và hoạt động liên tục 24/7, qua đó hỗ trợ cả thí sinh và sinh viên tiếp cận thông tin kịp thời, đồng thời giảm tải áp lực cho hệ thống nhân sự của các cơ sở giáo dục.

1.2 Xác định bài toán và các vấn đề tồn tại

Thực tiễn công tác tư vấn tuyển sinh và hỗ trợ sinh viên hiện nay đang đối mặt với nhiều thách thức đáng kể, chủ yếu xuất phát từ đặc điểm của dữ liệu và hạn chế trong cách thức khai thác thông tin.

Thứ nhất, dữ liệu mang tính phân mảnh cao và thiếu tính hệ thống. Các thông tin liên quan đến tuyển sinh, chương trình đào tạo, quy chế học vụ hay học phí không được tập trung tại một nguồn thống nhất mà phân tán trên website của từng trường, các văn

bản đề án dưới dạng PDF dài và các kênh truyền thông trực tuyến thiếu cơ chế kiểm chứng. Đặc biệt trong năm 2025, với sự xuất hiện của hơn 20 phương thức xét tuyển khác nhau cùng những điều chỉnh liên tục về quy chế, việc tra cứu và đối chiếu thông tin theo phương thức thủ công trở nên khó khăn và tốn nhiều thời gian. Không chỉ thí sinh, mà ngay cả sinh viên đang theo học cũng gặp trở ngại trong việc tiếp cận thông tin học vụ cập nhật, từ đó làm gia tăng nguy cơ tiếp nhận dữ liệu sai lệch hoặc lỗi thời.

Thứ hai, hạn chế của các phương thức hỗ trợ hiện có. Các kênh tư vấn truyền thống phụ thuộc vào nhân sự con người không thể đáp ứng nhu cầu truy vấn 24/7 với lưu lượng lớn. Trong khi đó, các hệ thống chatbot thế hệ cũ (rule-based) bộc lộ rõ nhược điểm về kỹ thuật: hoạt động dựa trên từ khóa cứng nhắc, khả năng xử lý ngữ cảnh tiếng Việt kém và không có cơ chế cập nhật dữ liệu thời gian thực (real-time). Những hạn chế này đặt ra yêu cầu về một giải pháp ứng dụng mô hình ngôn ngữ lớn kết hợp tìm kiếm tự động để nâng cao hiệu quả tư vấn.

1.3 Mục tiêu và Phạm vi nghiên cứu

Nhằm giải quyết bài toán cung cấp thông tin tuyển sinh và học vụ một cách chính xác, tự động và kịp thời, đề tài đặt ra mục tiêu cốt lõi là xây dựng một hệ thống Chatbot tư vấn hoạt động trên nền tảng Mô hình ngôn ngữ nhỏ (SLM) được tinh chỉnh chuyên sâu. Hệ thống sẽ đóng vai trò trợ lý ảo, có khả năng:

- **Hiểu và tương tác bằng ngôn ngữ tự nhiên tiếng Việt:** Xử lý linh hoạt các câu hỏi đa dạng về quy chế, điểm chuẩn, học phí, ngành học, thông tin chung,...
- **Cung cấp thông tin chính xác và cập nhật:** Tích hợp kỹ thuật RAG với cơ sở dữ liệu Vector và Web Search, đảm bảo nguồn tin cậy và dữ liệu thời gian thực.
- **Tối ưu hóa trải nghiệm người dùng:** Đảm bảo tốc độ phản hồi nhanh và tính nhất quán, hữu ích cho học sinh, phụ huynh.

Về phạm vi, nghiên cứu tập trung vào dữ liệu tuyển sinh và học vụ của các trường đại học lớn tại Việt Nam, cũng như các quy chế chung của Bộ Giáo dục và Đào tạo. Đối tượng sử dụng chính là sinh viên và học sinh THPT tuyển sinh 2026+. Về mặt công nghệ, trọng tâm là phát triển các giải pháp phần mềm ứng dụng NLP và RAG, giới hạn nghiên cứu ở cấp độ ứng dụng mà không đi sâu vào tối ưu hóa hạ tầng phần cứng.

1.4 Ý nghĩa khoa học và thực tiễn của đề tài

Kết quả của nghiên cứu mang lại những đóng góp giá trị trên phương diện khoa học thông qua việc ứng dụng các kỹ thuật tiên tiến để giải quyết bài toán xử lý ngôn ngữ tiếng Việt trong miền dữ liệu giáo dục đặc thù. Đồng thời, quá trình xây dựng hệ thống cũng cung cấp những kinh nghiệm thực nghiệm quý báu trong việc tối ưu hóa mô hình

hỏi đáp tự động, xử lý nhiều dữ liệu và tích hợp tri thức đa nguồn. Những kết quả này có thể làm tiền đề cho các nghiên cứu xa hơn về các hệ thống gợi ý chuyên sâu hoặc các trợ lý ảo phục vụ cá nhân hóa trong giáo dục.

Về mặt thực tiễn, sản phẩm của đề tài mang lại lợi ích kép cho xã hội và hệ thống giáo dục. Đối với thí sinh và sinh viên, đây là một công cụ tra cứu miễn phí, tin cậy và hoạt động liên tục, giúp xóa bỏ rào cản tiếp cận thông tin giữa các vùng miền. Đối với các cơ sở giáo dục, hệ thống hỗ trợ nâng cao chất lượng dịch vụ công, tiết kiệm chi phí nhân sự và chuyên nghiệp hóa công tác tuyển sinh trong kỷ nguyên số. Việc triển khai thành công hệ thống sẽ góp phần thúc đẩy quá trình chuyển đổi số trong giáo dục, hướng tới một môi trường tuyển sinh minh bạch và hiệu quả hơn.

1.5 Bố cục báo cáo

Báo cáo tổng kết đồ án được tổ chức thành 5 chương với nội dung chính như sau:

- **Chương 1:** Trình bày bối cảnh, bài toán, mục tiêu, phạm vi và ý nghĩa của đề tài hệ thống chatbot hỗ trợ sinh viên và tư vấn tuyển sinh.
- **Chương 2:** Giới thiệu cơ sở lý thuyết về chatbot trong giáo dục, mô hình ngôn ngữ Qwen3-4B, các kỹ thuật tinh chỉnh (LoRA, RAFT, RLHF/DPO) và kiến trúc RAG kết hợp Web Search.
- **Chương 3:** Mô tả chi tiết kiến trúc hệ thống, các pipeline Local Search và Web Search, cơ chế định tuyến truy vấn và quy trình triển khai hệ thống chatbot.
- **Chương 4:** Trình bày hệ thống đánh giá, các thiết lập thực nghiệm và kết quả đo lường chất lượng mô hình.
- **Chương 5:** Tổng kết những kết quả đạt được, các vấn đề kỹ thuật đã xử lý và đề xuất hướng phát triển trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về hệ thống Chatbot trong giáo dục

Trong bối cảnh chuyển đổi số mạnh mẽ hiện nay, việc ứng dụng Trí tuệ nhân tạo (Artificial Intelligence – AI) vào các dịch vụ công đang trở thành xu hướng tất yếu, trong đó lĩnh vực giáo dục là một trong những lĩnh vực được quan tâm đặc biệt. Về mặt kỹ thuật, chatbot (hay tác tử hội thoại) được định nghĩa là hệ thống phần mềm có khả năng tương tác và duy trì hội thoại với con người thông qua ngôn ngữ tự nhiên (Natural Language Processing – NLP). Mục tiêu cốt lõi của chatbot là tự động hóa các quy trình hỗ trợ, giúp người dùng tiếp cận thông tin một cách nhanh chóng, liên tục và không phụ thuộc vào sự hiện diện trực tiếp của nhân sự.

Trong bối cảnh giáo dục đại học, chatbot đã chứng minh vai trò quan trọng trong việc giải quyết bài toán quá tải thông tin, đặc biệt vào mùa tuyển sinh. Thay vì chỉ dừng lại ở việc trả lời các câu hỏi thường gặp (Frequently Asked Questions – FAQs) liên quan đến thời gian hay thủ tục, các hệ thống chatbot hiện đại đang dần phát triển thành những “trợ lý ảo thông minh”. Các hệ thống này không chỉ góp phần giảm tải áp lực hành chính cho các đơn vị đào tạo mà còn nâng cao trải nghiệm của người học thông qua khả năng phản hồi 24/7 và hỗ trợ tư vấn mang tính cá nhân hóa.

Sự phát triển mạnh mẽ của các Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs) như GPT, Claude hay Qwen đã tạo ra bước ngoặt quan trọng, đánh dấu sự chuyển dịch từ các chatbot dựa trên kịch bản (rule-based) sang thế hệ chatbot sinh nội dung (Generative AI). Nhờ khả năng thấu hiểu ngữ cảnh sâu (*deep contextual understanding*) và suy luận logic, các chatbot thế hệ mới có thể xử lý những truy vấn phức tạp như so sánh ngành học, phân tích cơ hội việc làm hoặc tư vấn lựa chọn trường dựa trên nhiều tiêu chí khác nhau. Đặc biệt, khi được tích hợp với các kỹ thuật truy hồi thông tin tiên tiến như *semantic search* và cơ sở dữ liệu vector, chatbot có khả năng khai thác tri thức chính xác từ dữ liệu nội bộ của nhà trường cũng như các nguồn thông tin bên ngoài, qua đó nâng cao độ tin cậy và tính cập nhật của câu trả lời.

Những tiến bộ này tạo nền tảng cho việc xây dựng các hệ thống chatbot tư vấn tuyển sinh thông minh, kết hợp giữa mô hình ngôn ngữ lớn và kiến trúc truy hồi tri thức, là

hướng tiếp cận được lựa chọn trong dự án này.

2.2 Mô hình ngôn ngữ và kỹ thuật huấn luyện

2.2.1 Mô hình nền tảng Qwen3-4B

Hệ thống chatbot trong đề án được xây dựng dựa trên mô hình ngôn ngữ **Qwen3-4B**, một mô hình thuộc kiến trúc *Causal Transformer* với quy mô xấp xỉ 4 tỷ tham số. Qwen3-4B được thiết kế theo hướng cân bằng giữa hiệu năng suy luận và chi phí tính toán, phù hợp với các kịch bản triển khai thực tế trên hạ tầng GPU phổ biến.

Về mặt kiến trúc, Qwen3-4B bao gồm 36 lớp Transformer và áp dụng cơ chế *Grouped Query Attention (GQA)* nhằm tối ưu hóa hiệu quả bộ nhớ trong quá trình suy luận. Cụ thể, mô hình sử dụng 32 đầu truy vấn (*Q-heads*) và 8 đầu khóa-giá trị (*KV-heads*), cho phép giảm đáng kể dung lượng *KV Cache* mà vẫn duy trì chất lượng biểu diễn ngữ nghĩa. Đặc tính này đặc biệt quan trọng đối với các hệ thống hội thoại yêu cầu xử lý đồng thời nhiều phiên làm việc.

Về khả năng xử lý ngữ cảnh, Qwen3-4B hỗ trợ độ dài ngữ cảnh mặc định lên tới 32,768 tokens, cho phép mô hình tiếp nhận và phân tích các tài liệu tuyển sinh dài trong một lần suy luận. Bên cạnh đó, mô hình hỗ trợ kỹ thuật *YaRN (Yet another RoPE N-dimensional scaling)*, giúp mở rộng khả năng đọc hiểu các chuỗi văn bản dài hơn giới hạn huấn luyện ban đầu khi cần thiết.

Nền tảng tri thức của Qwen3-4B được hình thành thông qua quá trình tiền huấn luyện (*pre-training*) trên tập dữ liệu quy mô lớn khoảng 36 nghìn tỷ tokens, bao phủ hơn 100 ngôn ngữ khác nhau. Nhờ chiến lược huấn luyện nhiều giai đoạn, mô hình hỗ trợ hai chế độ vận hành linh hoạt:

- **Chế độ suy luận (Thinking Mode):** Tăng cường khả năng lập luận logic thông qua việc sinh các bước suy nghĩ trung gian, phù hợp với các truy vấn phân tích và so sánh phức tạp.
- **Chế độ phản hồi nhanh (Non-thinking Mode):** Tối ưu hóa cho tốc độ suy luận, đáp ứng các tác vụ hội thoại và tra cứu thông tin thông thường với độ trễ thấp.

Nhờ sự kết hợp giữa kiến trúc attention hiệu quả, khả năng đa ngôn ngữ mạnh mẽ và hỗ trợ ngữ cảnh dài, nhóm đã chọn Qwen3-4B cho bài toán chatbot tư vấn tuyển sinh.

2.2.2 Kỹ thuật LORA

Để vượt qua rào cản về tài nguyên GPU và lưu trữ của phương pháp tinh chỉnh toàn phần (*full fine-tuning*) trên các mô hình ngôn ngữ lớn, Hu et al. [2] đã đề xuất **LoRA (Low-Rank Adaptation)**. Kỹ thuật này được xây dựng trên giả thuyết *số chiều nội tại*

(*intrinsic dimension*), khẳng định rằng quá trình thích ứng nhiệm vụ thực chất chỉ diễn ra trong một không gian con hạng thấp. Bằng cách cố định trọng số tiền huấn luyện và chỉ tối ưu hóa các ma trận phân rã hạng thấp, LoRA giảm thiểu đáng kể chi phí tính toán và bộ nhớ mà vẫn duy trì hiệu năng tương đương mô hình gốc.

Nguyên lý hoạt động

Thay vì cập nhật trực tiếp ma trận trọng số tiền huấn luyện $W_0 \in \mathbb{R}^{d \times k}$, LoRA mô hình hóa ma trận cập nhật ΔW thông qua phân rã hạng thấp:

$$\Delta W = BA,$$

trong đó $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ và $r \ll \min(d, k)$.

Quá trình truyền thuận với đầu vào x được định nghĩa như sau:

$$h = W_0 x + BAx. \tag{1}$$

Trong đó, ma trận W_0 được đóng băng hoàn toàn trong suốt quá trình huấn luyện, chỉ các ma trận A và B được cập nhật gradient. Với cấu hình điển hình $d = 4096$ và $r = 8$, phương pháp này giúp giảm số lượng tham số cần huấn luyện xuống hàng trăm lần so với tinh chỉnh toàn phần.

Để đảm bảo tính ổn định trong giai đoạn đầu huấn luyện, LoRA áp dụng chiến lược khởi tạo trong đó ma trận A được khởi tạo theo phân phối Gaussian, còn ma trận B được khởi tạo bằng không ($B = 0$), đảm bảo $\Delta W = 0$ tại thời điểm bắt đầu. Ngoài ra, một hệ số tỷ lệ α/r được sử dụng nhằm kiểm soát độ lớn của cập nhật và giảm sự phụ thuộc của siêu tham số vào việc lựa chọn hạng r .

Ưu điểm của LORA trong triển khai hệ thống

So với các phương pháp tinh chỉnh hiệu quả tham số khác như Adapter Layers hay Prefix Tuning, LoRA mang lại ba lợi thế kỹ thuật vượt trội:

- **Triệt tiêu độ trễ suy luận:** Nhờ tính chất tuyến tính, các tham số LoRA có thể được gộp trực tiếp vào trọng số gốc sau huấn luyện theo công thức

$$W_{\text{new}} = W_0 + BA,$$

qua đó đảm bảo tốc độ phản hồi thời gian thực trong giai đoạn suy luận mà không làm thay đổi kiến trúc mô hình hay phát sinh thêm chi phí tính toán.

- **Tối ưu hóa tài nguyên VRAM:** Việc chỉ cập nhật một tập con nhỏ các tham số hạng thấp giúp loại bỏ nhu cầu lưu trữ gradient cho toàn bộ mô hình, cho phép

huấn luyện và tinh chỉnh mô hình Qwen3-4B ngay trên hạ tầng phần cứng phổ thông, chẳng hạn như GPU NVIDIA T4 với 16GB bộ nhớ.

- **Chuyển đổi tác vụ linh hoạt:** Hệ thống có thể hỗ trợ nhiều tác vụ khác nhau bằng cách hoán đổi nhanh các module LoRA hạng nhẹ trên cùng một mô hình nền. Cách tiếp cận này giúp tiết kiệm đáng kể chi phí lưu trữ và tạo điều kiện thuận lợi cho việc mở rộng hoặc cập nhật chức năng của hệ thống trong tương lai.

2.2.3 Kỹ thuật RAFT

Tồn tại sự khác biệt cơ bản giữa cơ chế huấn luyện SFT truyền thống (thiên về ghi nhớ tri thức – *closed-book*) và thực tế vận hành RAG (yêu cầu trích xuất và suy luận – *open-book*). Đặc biệt trong bối cảnh tìm kiếm Web, kết quả trả về thường hỗn tạp, chứa cả tài liệu chính xác (*Oracle documents*) lẫn thông tin nhiễu (*Distractor documents*) như dữ liệu lỗi thời hoặc sai lệch. Để khắc phục nhược điểm dễ gây ảo giác của mô hình SFT trước dữ liệu nhiễu, Zhang et al. (2023) [5] đã đề xuất **RAFT (Retrieval Augmented Fine Tuning)** nhằm tối ưu hóa chuyên biệt khả năng lọc và trích xuất thông tin trong môi trường dữ liệu không hoàn hảo.

Nguyên lý hoạt động

Thay vì chỉ huấn luyện mô hình trả lời câu hỏi dựa trên tri thức nội tại, RAFT tái cấu trúc dữ liệu huấn luyện để mô phỏng lại bối cảnh truy xuất không hoàn hảo của thực tế. Mỗi mẫu dữ liệu huấn luyện được xây dựng dưới dạng bộ ba (Q, D, A^*) , bao gồm:

- Q : Câu hỏi của người dùng.
- D : Tập hợp các tài liệu tham khảo ngữ cảnh.
- A^* : Câu trả lời đích kèm theo chuỗi suy luận (Chain-of-Thought).

Điểm đột phá nằm ở cấu trúc của tập tài liệu D . Tập hợp này bao gồm hai loại thành phần được trộn lẫn:

1. **Tài liệu Oracle (D^*):** Các đoạn văn bản chứa thông tin chính xác để trả lời câu hỏi.
2. **Tài liệu Distractor ($D_{distractor}$):** Các đoạn văn bản không liên quan hoặc gây nhiễu (ví dụ: Quy chế tuyển sinh năm 2023 khi câu hỏi là năm 2025).

Mục tiêu tối ưu hóa của mô hình là cực đại hóa xác suất sinh ra câu trả lời A^* dựa trên ngữ cảnh hỗn hợp này:

$$\mathcal{L}_{RAFT} = - \sum \log P(A^* \mid Q, D^* \cup D_{distractor})$$

Thông qua cơ chế này, mô hình Qwen3-4B không chỉ học kiến thức về tuyển sinh mà còn hình thành kỹ năng tư duy phản biện: định danh chính xác đoạn văn chứa thông tin hữu ích (D^*) và chủ động phớt lờ các đoạn tin nhiễu ($D_{distractor}$).

Ưu điểm của RAFT trong triển khai hệ thống

- **Độ bền vững trước nhiễu (Robustness):** Giúp mô hình chủ động phân loại và loại bỏ các thông tin lỗi thời hoặc không liên quan (*distractors*) từ kết quả tìm kiếm Web hỗn tạp, đảm bảo tính chính xác dựa trên nguồn dữ liệu chính thống.
- **Tính giải thích được (Explainability):** Khuyến khích cơ chế trích dẫn nguyên văn (*verbatim quoting*) từ tài liệu gốc thay vì sinh văn bản ngẫu nhiên, cho phép người dùng dễ dàng đối chiếu và kiểm chứng nguồn thông tin.
- **Thích ứng miền dữ liệu (Domain Adaptation):** Tối ưu hóa khả năng xử lý cấu trúc ngôn ngữ đặc thù của văn bản hành chính giáo dục, giúp trích xuất chính xác các thực thể chi tiết như mã ngành hay tổ hợp xét tuyển.

2.2.4 Kỹ thuật DPO

Phương pháp Direct Preference Optimization (DPO) được đề xuất bởi Rafailov et al. [5] như một biến thể đơn giản hóa của RLHF, loại bỏ nhu cầu huấn luyện mô hình phần thưởng riêng biệt.

Nguyên lý hoạt động

Khác với phương pháp RLHF truyền thống sử dụng thuật toán PPO để tối ưu hóa chính sách dựa trên một mô hình phần thưởng (reward model) riêng biệt, Direct Preference Optimization (DPO) tái định nghĩa bài toán căn chỉnh mô hình dưới dạng bài toán phân loại nhị phân trực tiếp trên dữ liệu xếp hạng. Thay vì duy trì và huấn luyện thêm một reward model, DPO tối ưu hóa trực tiếp mô hình ngôn ngữ bằng cách cực đại hóa chênh lệch logit giữa phản hồi được lựa chọn (y_{chosen}) và phản hồi bị loại bỏ (y_{rejected}).

Cụ thể, với mỗi bộ dữ liệu dạng $(x, y_{\text{chosen}}, y_{\text{rejected}})$, trong đó x là prompt đầu vào, hàm mất mát của DPO được định nghĩa như sau:

$$\mathcal{L}_{\text{DPO}} = - \sum_{(x, y_{\text{chosen}}, y_{\text{rejected}})} \log \sigma(f_{\theta}(x, y_{\text{chosen}}) - f_{\theta}(x, y_{\text{rejected}})), \quad (2)$$

trong đó $f_{\theta}(\cdot)$ biểu diễn logit (hoặc log-likelihood) do mô hình ngôn ngữ sinh ra và $\sigma(\cdot)$ là hàm sigmoid.

Thông qua cơ chế này, mô hình ngôn ngữ tự đóng vai trò là mô hình phần thưởng ngầm định, giúp loại bỏ hoàn toàn quy trình huấn luyện reward model và bước lấy mẫu phản hồi phức tạp như trong PPO-based RLHF. Nhờ đó, DPO không chỉ đơn giản hóa

pipeline huấn luyện mà còn cải thiện đáng kể tính ổn định và khả năng tái lập của quá trình căn chỉnh mô hình [?].

Ưu điểm của DPO trong triển khai hệ thống

Direct Preference Optimization (DPO) đơn giản hóa đáng kể quy trình RLHF truyền thống bằng cách loại bỏ hoàn toàn mô hình phần thưởng và thuật toán PPO. Thay vào đó, mô hình ngôn ngữ được tối ưu trực tiếp trên dữ liệu so sánh cặp phản hồi (chosen/rejected) thông qua hàm mất mát log-likelihood, giúp quá trình huấn luyện ổn định và dễ kiểm soát [5].

Nhờ không yêu cầu sampling trong vòng lặp huấn luyện, DPO giảm chi phí tính toán và tránh hiện tượng mất ổn định thường gặp trong RLHF. Đồng thời, phương pháp này tương thích tự nhiên với các kỹ thuật fine-tuning hiệu quả tham số như LoRA, cho phép triển khai và cập nhật mô hình chatbot quy mô lớn một cách linh hoạt và tiết kiệm tài nguyên.

• Pipeline huấn luyện (LoRA + RAFT + DPO):

1. **LoRA:** Giúp mô hình nền tảng thích nghi với miền dữ liệu tuyển sinh với chi phí thấp.
2. **RAFT:** Trang bị cho mô hình khả năng xử lý ngữ cảnh RAG và lọc nhiễu từ kết quả tìm kiếm Web.
3. **DPO:** Đóng vai trò alignment, điều chỉnh hành vi của mô hình khớp với mong đợi của con người, giảm thiểu tối đa các câu trả lời ảo giác hoặc thiếu an toàn.

2.2.5 Tối ưu hóa suy luận với vLLM

Dự án sử dụng thư viện **vLLM** [7], một hệ thống phục vụ mô hình ngôn ngữ hiệu suất cao được thiết kế dựa trên thuật toán cốt lõi **PagedAttention**.

Cơ chế hoạt động của PagedAttention

Lấy cảm hứng từ kỹ thuật quản lý bộ nhớ ảo và phân trang (Paging) trong các hệ điều hành kinh điển, PagedAttention cho phép lưu trữ các KV Cache trong các khối bộ nhớ không liên tiếp (*non-contiguous memory blocks*). Thay vì cấp phát một mảng lớn cố định, vLLM chia nhỏ KV Cache của mỗi chuỗi thành các khối (blocks) và ánh xạ chúng linh hoạt vào bộ nhớ vật lý của GPU thông qua một bảng trang (block table). Cơ chế này giúp loại bỏ gần như hoàn toàn sự lãng phí bộ nhớ do phân mảnh, cho phép tận dụng tối đa dung lượng VRAM hiện có.

Hiệu quả đối với hệ thống Chatbot

Việc tích hợp vLLM mang lại hai lợi thế quyết định cho hệ thống tư vấn tuyển sinh:

- **Tăng thông lượng (Throughput) đột phá:** Nhờ quản lý bộ nhớ hiệu quả, hệ thống có thể thực hiện *Continuous Batching* – xử lý đồng thời hàng chục yêu cầu của học sinh cùng lúc trên cùng một GPU thay vì phải xếp hàng chờ đợi, giúp giải quyết bài toán quá tải trong mùa cao điểm tuyển sinh.
- **Hỗ trợ ngữ cảnh dài (Long Context):** Với mô hình Qwen3-4B và kiến trúc RAG, độ dài ngữ cảnh đầu vào thường rất lớn (do chứa nhiều tài liệu tham khảo). vLLM giúp hệ thống duy trì hoạt động ổn định, tránh lỗi tràn bộ nhớ (OOM) khi phải xử lý các văn bản đề án dài hàng chục trang.

2.3 Kỹ thuật tìm kiếm và tăng cường tri thức (RAG & Search)

Để khắc phục nhược điểm về dữ liệu tĩnh và hiện tượng *hallucination* của các mô hình ngôn ngữ, hệ thống áp dụng kiến trúc **Retrieval-Augmented Generation (RAG)** kết hợp với cơ chế tìm kiếm Web. Phần này trình bày các cơ sở lý thuyết nền tảng của các thành phần liên quan.

2.3.1 Tổng quan kiến trúc RAG

RAG là kiến trúc lai ghép giữa bộ nhớ tham số (*Parametric Memory* – trọng số của mô hình ngôn ngữ) và bộ nhớ phi tham số (*Non-parametric Memory* – cơ sở dữ liệu bên ngoài) [1]. Thay vì yêu cầu mô hình phải ghi nhớ toàn bộ thông tin như điểm chuẩn hay chỉ tiêu tuyển sinh – vốn thay đổi theo từng năm – RAG cho phép truy xuất thông tin liên quan từ kho dữ liệu bên ngoài tại thời điểm suy luận, sau đó sử dụng thông tin này làm ngữ cảnh để sinh câu trả lời.

Cách tiếp cận này giúp hệ thống duy trì tính cập nhật, minh bạch nguồn thông tin và giảm thiểu hiện tượng sinh nội dung sai lệch.

2.3.2 Biểu diễn vector và tìm kiếm tương đồng

Nền tảng của tìm kiếm thông tin cục bộ (*Local Search*) trong RAG là kỹ thuật biểu diễn văn bản dưới dạng vector nhúng (*Embeddings*).

- **Mô hình nhúng (Embedding Model):** Các đoạn văn bản được chia nhỏ (chunking) với kích thước 512 tokens và độ chồng lấp 64 tokens từ đề án tuyển sinh và tài liệu liên quan, sau đó đưa qua mô hình nhúng đa ngôn ngữ *multilingual-e5-small* để ánh xạ thành các vector 384 chiều trong không gian embedding. Các đoạn văn có ngữ nghĩa tương đồng sẽ nằm gần nhau trong không gian vector này.
- **Cơ sở dữ liệu vector:** Hệ thống sử dụng thư viện FAISS (Facebook AI Similarity Search) với thuật toán chỉ mục IndexFlatL2 để thực hiện tìm kiếm láng giềng gần nhất chính xác (Exact Nearest Neighbor Search) dựa trên khoảng cách L2. FAISS

cho phép tìm kiếm nhanh và chính xác ngay cả trên tập dữ liệu lớn với hơn 50,000 vector chunks từ 200+ trường đại học.

2.3.3 Xếp hạng lại ngữ nghĩa bằng Cross-Encoder

Việc chỉ dựa vào so sánh vector theo mô hình Bi-Encoder thường chưa đủ độ chính xác do hiện tượng mất mát thông tin khi nén ngữ nghĩa. Để cải thiện chất lượng tìm kiếm, đồ án áp dụng bước xếp hạng lại (*Reranking*) sử dụng mô hình Cross-Encoder, cụ thể là *BGE-M3* [6].

Khác với Bi-Encoder xử lý câu hỏi và văn bản một cách độc lập, Cross-Encoder nhận đầu vào là cặp (Câu hỏi, Văn bản) và cho phép các token tương tác trực tiếp thông qua cơ chế Self-Attention đầy đủ. Mô hình từ đó đánh giá mức độ liên quan dựa trên sự tương tác ngữ nghĩa sâu, đóng vai trò như một “bộ lọc tinh” để loại bỏ các kết quả nhiễu, đặc biệt quan trọng trong bối cảnh tìm kiếm Web.

2.3.4 Tìm kiếm Web và định tuyến truy vấn

Đối với các truy vấn mang tính thời sự cao như “thông tin tuyển sinh mới nhất”, cơ sở dữ liệu tĩnh là không đủ. Do đó, hệ thống tích hợp khả năng tìm kiếm Web dựa trên quy trình tác tử (*Agentic Workflow*).

- **Mở rộng truy vấn (Query Expansion):** Mô hình ngôn ngữ được sử dụng để chuyển đổi câu hỏi tự nhiên của người dùng thành các từ khóa phù hợp với công cụ tìm kiếm.
- **Lọc trang (Page Filtering):** Sử dụng BAAI/bge-reranker-v2-m3 để đọc lướt tiêu đề và đoạn trích của kết quả tìm kiếm, từ đó loại bỏ các trang quảng cáo hoặc không liên quan trước khi thu thập nội dung chi tiết.

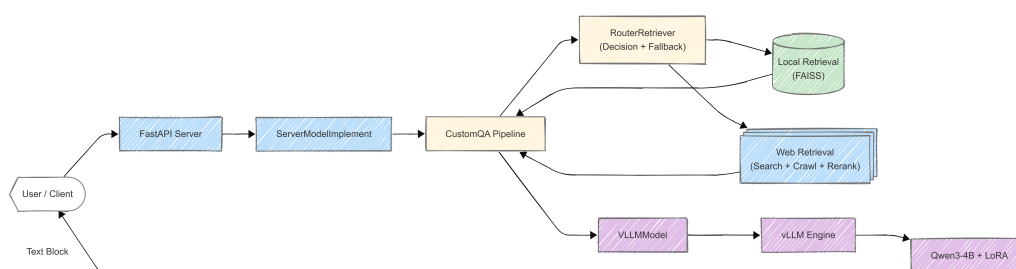
Chương 3

GIẢI PHÁP ĐỀ XUẤT VÀ TRIỂN KHAI

3.1 Tổng quan kiến trúc hệ thống Chatbot

Hệ thống chatbot được thiết kế theo kiến trúc *backend service* gồm ba lớp chính:

- Lớp mô hình sinh ngôn ngữ (SLM) với vLLM.
- Lớp truy xuất tri thức (RAG: Web Search + Local Search).
- Lớp server API phục vụ client.



Hình 3.1: Kiến trúc hệ thống Chatbot

3.1.1 Các thành phần chính

Ở mức logic, hệ thống gồm các thành phần:

- **Mô hình ngôn ngữ Qwen3-4B (LoRA + RAFT):** được triển khai trên nền vLLM, hỗ trợ suy luận hội thoại và được tinh chỉnh bằng LoRA kết hợp RAFT để khai thác hiệu quả ngữ cảnh truy xuất. Thành phần này sinh câu trả lời cuối cùng cho người dùng.
- **Web Search Pipeline:** gói trong lớp WebRetriever và DataRetrieverPipeline, chịu trách nhiệm sinh truy vấn, gọi các API tìm kiếm (Google, Brave), crawl nội dung, chia nhỏ văn bản, suy luận điểm và rerank bằng mô hình nhỏ (Cross-Encoder bge-reranker-v2-m3), sau đó trả về các đoạn ngữ cảnh liên quan nhất từ Web.

- **Local Search Pipeline:** được hiện thực bởi `LocalRetriever`, tìm kiếm ngữ nghĩa trên kho dữ liệu tĩnh `local.pkl` bằng embedding + FAISS. Kênh này phục vụ các câu hỏi dựa trên tài liệu nội bộ đã chuẩn hoá.
- **Router và Chunk Ranker:** lớp `RouterRetriever` nhận câu hỏi và cấu hình tham số, quyết định sử dụng Web Search hay Local Search. Thành phần `ChunkRanker` dùng Cross-Encoder để chấm điểm và lọc các đoạn ngữ cảnh (chunks) trước khi đưa vào mô hình Reader, đảm bảo chỉ giữ lại những đoạn thực sự liên quan.
- **Lớp điều phối QA (CustomQA):** bao gói toàn bộ pipeline RAG: gọi `RouterRetriever` để lấy ngữ cảnh, format thành prompt theo template (`READER_TEMPLATE`) và gọi Qwen3-4B LoRA để sinh câu trả lời.
- **Server API và tích hợp ngrok:** lớp `ServerModelImplement` triển khai giao diện `ServerModel` (hàm `pre_inference` và `inference`), được gói vào một ứng dụng FastAPI thông qua `construct_app`, sau đó publish ra ngoài bằng `uvicorn` và `ngrok`. Client (web/chat UI) chỉ cần gửi request HTTP tới domain ngrok để sử dụng chatbot.

3.1.2 Luồng dữ liệu tổng quát

Luồng xử lý một câu hỏi người dùng có thể tóm tắt như sau:

1. **Client gửi câu hỏi** tới server (FastAPI) kèm các tham số sinh (`GenerationParams: use_websearch, use_localdb, max_query, k_docs, ...`).
2. **Bước tiền suy luận (pre-inference):**
 - (a) `CustomQA.pre_inference` gọi `RouterRetriever.retrieve` để: (i) Router sinh truy vấn con, (ii) chạy Web Search hoặc Local Search, (iii) rerank và lọc các đoạn văn bản liên quan.
 - (b) Các *RAG chunks* được format lại thành ngữ cảnh thống nhất để tạo ra prompt đầy đủ: *Ngữ cảnh đã lọc + câu hỏi gốc + chỉ dẫn trả lời*.
3. **Bước suy luận (inference):** prompt này được gửi vào mô hình Qwen3-4B (LoRA Reader) chạy trên vLLM. Mô hình sinh câu trả lời theo dạng streaming và trả về cho client qua API.
4. **Lưu trữ và logging:** sau khi kết thúc suy luận, hệ thống lưu lại thông tin: tham số sinh, các nguồn RAG đã dùng (web/local) và câu trả lời cuối cùng, phục vụ phân tích và cải tiến về sau.

Với kiến trúc này, chatbot vừa tận dụng được sức mạnh của mô hình ngôn ngữ lớn, vừa kết hợp chặt chẽ với hai nguồn tri thức Web và Local, thông qua một lớp Router linh hoạt và cơ chế RAG đa tầng để đảm bảo câu trả lời có căn cứ, cập nhật và phù hợp ngữ cảnh tuyến sinh.

3.2 Xử lý dữ liệu và huấn luyện mô hình

Để mô hình Qwen3-4B có thể trả lời chính xác các câu hỏi về tuyển sinh, học vụ và thông tin công khai của các trường đại học Việt Nam, nhóm nghiên cứu đã thiết kế quy trình xử lý dữ liệu chặt chẽ nhằm tạo ra bộ dữ liệu RAFT chất lượng cao.

3.2.1 Xử lý dữ liệu

Hệ thống sử dụng hai file markdown chứa template câu hỏi làm nguồn dữ liệu gốc: `hoc_vu.md` (~350 templates về tuyển sinh và học vụ) và `thong_tin_cong_khai_kiem_dinh.md` (~118 templates về thông tin công khai và kiểm định). Tổng cộng có khoảng 468 template câu hỏi, bao phủ đầy đủ ba nhiệm vụ chính của chatbot: tư vấn tuyển sinh, hỗ trợ sinh viên đang học, và tra cứu thông tin công khai.

Quy trình sinh dữ liệu tự động

Quy trình được thực hiện qua script `generateRAFT_data_gpt.py` với các bước thiết kế chính:

1. **Trích xuất template câu hỏi:** Hệ thống đọc và phân tích hai file markdown, trích xuất các template câu hỏi cùng với metadata về danh mục (category) để theo dõi phân bố dữ liệu.
2. **Thay thế placeholder và tạo biến thể:** Với mỗi template, hệ thống thay thế các placeholder (`[Tên trường]`, `[Tên ngành]`, `[Năm]`, ...) bằng giá trị cụ thể từ danh sách ~200+ trường đại học (load từ `school_alias.json` và `school_name.json`) và các ngành học phù hợp. Để đảm bảo tính đa dạng và tự nhiên của câu hỏi, hệ thống áp dụng hai kỹ thuật:
 - *Biến thể dựa trên pattern matching:* Tạo 5 biến thể cho mỗi template bằng cách thay đổi cách diễn đạt (ví dụ: "Điểm chuẩn ngành X là bao nhiêu?" → "Ngành X có điểm chuẩn bao nhiêu?", "Mức điểm chuẩn của ngành X?").
 - *Biến thể dựa trên GPT paraphrasing:* Sử dụng mô hình GPT-4o-mini để tạo 3 biến thể tự nhiên cho mỗi template, đảm bảo 90–95% câu hỏi có biến thể tự nhiên (tỷ lệ paraphrase mặc định: $r_p = 0.5$). Kỹ thuật này giúp câu hỏi không quá cứng nhắc theo template mà vẫn đảm bảo tính đầy đủ.
3. **Lọc và giới hạn số lượng:** Hệ thống loại bỏ các câu hỏi trùng lặp và giới hạn tổng số câu hỏi về mục tiêu (~6000 câu), đảm bảo phân bố đều giữa các danh mục.

Kết quả: Hệ thống sinh ra khoảng 5231 câu hỏi unique, đã được fix các placeholder và lưu vào file `questions_extracted_fixed.txt`.

3.2.2 Quy trình sinh câu trả lời chuẩn (Ground Truth Generation)

Sau khi có danh sách câu hỏi, hệ thống sử dụng script `generateRAFT_from_questions.py` để sinh câu trả lời chi tiết:

1. **Load câu hỏi:** Đọc danh sách câu hỏi từ file `questions_extracted_fixed.txt`.
2. **Sinh câu trả lời bằng GPT:** Với mỗi câu hỏi, hệ thống gọi API GPT-4o-mini (hoặc GPT-4o cho chất lượng cao hơn) với một `SYSTEM_PROMPT` chi tiết, bao gồm:
 - Ba nhiệm vụ chính: Tư vấn tuyển sinh, Hỗ trợ sinh viên đang học, Tra cứu thông tin công khai và kiểm định.
 - Format Markdown chuẩn: Tiêu đề (###), Tóm tắt (2–3 bullet với số liệu), Bảng chính (nếu có ≥ 5 số liệu), Nội dung chi tiết, Nguồn, Lưu ý, và Follow-up questions.
 - Quy tắc bắt buộc: Đảm bảo tính nhất quán, đầy đủ thông tin, và tuân thủ format.
3. **Validation và checkpointing:** Mỗi câu trả lời được validate về format và độ dài (tối thiểu 200 từ). Hệ thống lưu checkpoint định kỳ (mỗi 50 Q&A) để có thể resume khi gặp lỗi hoặc gián đoạn.

Kết quả: Hệ thống sinh ra ~ 5231 cặp Q&A chất lượng cao, lưu vào file `RAFT_dataset_from_questions`.

3.2.3 Quy trình Tích hợp RAFT (Data Engineering Pipeline)

Để huấn luyện với RAFT, hệ thống cần bổ sung thêm các tài liệu tham khảo (oracle và distractor documents) cho mỗi cặp Q&A. Quy trình này được thực hiện qua hai bước kỹ thuật:

Bước 1: Cập nhật Vector Database

Vector database ban đầu có thể thiếu hoặc cũ so với dữ liệu Q&A mới. Hệ thống sử dụng script `update_vector_db_complete.py` để:

1. Load documents hiện có từ `vector_database/vectordb/static.pkl` (khoảng 388 documents).
2. Tạo documents mới từ Q&A dataset: Với mỗi cặp Q&A, hệ thống extract thông tin từ `answer` để tạo thành một LangChain Document với metadata: `school_id` (Mã trường), `section` (Khu vực nội dung), `source`, và `type` ("qa_generated").

3. Kết hợp và loại trùng lặp: Merge documents cũ và mới, loại bỏ trùng lặp dựa trên `page_content`, ưu tiên documents mới.
4. Tạo lại FAISS vectorstore: Sử dụng embedding model `intfloat/multilingual-e5-small` để tạo embeddings và lập chỉ mục FAISS.
5. Lưu vào thư mục mới: `vector_database/vectordb_updated/`.

Kết quả: Vector database mới chứa ~ 800 documents.

Bước 2: Điền Oracle và Distractor Documents

Hệ thống sử dụng script `fill_raft_documents.py` để điền `oracle_doc` và `distractor_docs` vào mỗi record:

1. Load vector database: Đọc documents từ `vectordb_updated/static.pkl`.
2. Precompute document embeddings: Sử dụng `intfloat/multilingual-e5-small` để encode tất cả documents thành vectors (384 dimensions), normalize embeddings.
3. Với mỗi câu hỏi trong dataset:
 - Encode question thành embedding.
 - Tính cosine similarity giữa question embedding và tất cả document embeddings:

$$\text{similarity}(q, d) = \frac{q \cdot d}{\|q\| \|d\|} \quad (3)$$

- **Tìm Oracle document:** Document có similarity cao nhất (thường > 0.85).
- **Tìm Distractor documents:** Các documents có similarity trong khoảng $[0.6, 0.85]$ và không quá giống oracle ($\text{similarity} < 0.9$). Chọn top-2 distractors (có thể điều chỉnh qua `--num-distractors`).
- **Cơ chế xác suất:** 80% samples có oracle (điều chỉnh qua `--oracle-ratio`), 20% không có oracle để mô hình học cách trả lời khi không có context đầy đủ.

Cấu trúc dữ liệu RAFT cuối cùng: File `raft_dataset_final.jsonl` có định dạng:

```
{
  "question": "...",
  "answer": "...",
  "oracle_doc": {"content": "...", "source": "...", "metadata": {...}},
  "distractor_docs": [
    {"content": "...", "source": "...", "metadata": {...}},
    {"content": "...", "source": "...", "metadata": {...}}
  ]
}
```



```
],
"messages": [...],
}
```

Nhóm nghiên cứu áp dụng chiến lược tinh chỉnh hai giai đoạn nhằm tối ưu hóa từng khả năng cụ thể của mô hình.

3.2.4 Chiến lược Giai đoạn 1: Tinh chỉnh LoRA cơ bản

Mục tiêu giai đoạn này là để mô hình học cách trả lời câu hỏi theo format Markdown chuẩn. Dữ liệu huấn luyện không chứa context documents, mô hình chỉ học cách trả lời câu hỏi trực tiếp.

Cấu hình LoRA

Cấu hình LoRA được thiết lập như sau:

- **Rank (r):** 16 – Độ sâu của ma trận hạng thấp, cân bằng giữa khả năng học biểu diễn và số lượng tham số có thể huấn luyện.
- **Alpha (α):** 32 – Hệ số tỉ lệ, với $\alpha/r = 2.0$ nhằm điều chỉnh mức độ ảnh hưởng của các trọng số LoRA.
- **Target modules:** ["q_proj", "k_proj", "v_proj", "o_proj"] – Chỉ áp dụng LoRA cho các lớp Attention, không áp dụng cho các lớp MLP nhằm tối ưu hiệu quả huấn luyện.
- **Dropout:** $p = 0.05$ – Tỷ lệ dropout thấp để hạn chế hiện tượng underfitting.
- **Task type:** CAUSAL_LM – Phù hợp với bài toán sinh ngôn ngữ tự động.

Với cấu hình trên, số lượng tham số có thể huấn luyện chỉ chiếm khoảng 2–3% tổng số tham số của mô hình Qwen3-4B (khoảng 4 tỷ tham số), tương đương khoảng 80–120 triệu tham số.

Cấu hình huấn luyện

Cấu hình huấn luyện cho giai đoạn LoRA cơ bản được thiết lập như sau:

Tham số	Giá trị / Mô tả
Batch size	$b = 2$ trên mỗi thiết bị
Gradient accumulation	$g = 4$
Effective batch size	$B_{\text{eff}} = b \times g = 8$
Learning rate	$\eta = 2 \times 10^{-4}$ – Giá trị learning rate ổn định cho quá trình tinh chỉnh LoRA
Số epochs	$E = 2$ – Đủ để mô hình học được các mẫu (patterns) trong dữ liệu mà không gây hiện tượng overfitting
Max sequence length	$L_{\text{max}} = 512$ tokens – Phù hợp với các câu hỏi và câu trả lời ngắn, không bao gồm tài liệu ngữ cảnh
Mixed precision	bf16 – Sử dụng định dạng bfloat16 nhằm tiết kiệm bộ nhớ và tăng tốc độ huấn luyện
Optimizer	AdamW kết hợp với weight decay
Learning rate schedule	Cosine schedule với giai đoạn warmup

Bảng 3.1: Cấu hình huấn luyện cho giai đoạn LoRA cơ bản

Định dạng dữ liệu huấn luyện

Dữ liệu huấn luyện được chuẩn hóa theo template hội thoại của mô hình Qwen, với cấu trúc như sau:

```
<|im_start|>system
Bạn là một AI tư vấn tuyển sinh đại học chuyên
nghiệp...<|im_end|>
<|im_start|>user
Câu hỏi: ...<|im_end|>
<|im_start|>assistant
### Tiêu đề...
[Nội dung câu trả lời Markdown]
<|im_end|>
```

3.2.5 Chiến lược Giai đoạn 2: Tinh chỉnh LoRA + RAFT

Giai đoạn thứ hai sử dụng kỹ thuật **RAFT (Retrieval-Augmented Fine-Tuning)** kết hợp với LoRA. RAFT là phương pháp tinh chỉnh trong đó mỗi mẫu huấn luyện bao gồm câu hỏi, câu trả lời và các tài liệu tham khảo, bao gồm *oracle documents* (chứa thông tin chính xác) và *distractor documents* (chứa thông tin liên quan nhưng nhiễu). Mô hình được huấn luyện để phân biệt và khai thác thông tin từ oracle documents, đồng thời học cách bỏ qua các distractors trong quá trình sinh câu trả lời.

Phương pháp Hybrid Approach

Nhóm nghiên cứu áp dụng phương pháp *Hybrid Approach* để xử lý linh hoạt hai trường hợp dữ liệu huấn luyện:

- Nếu mẫu có context (oracle/distractor documents) $\rightarrow$ sử dụng format RAFT với context đầy đủ.
- Nếu mẫu không có context (insufficient context samples, chiếm khoảng $\sim 20\%$) $\rightarrow$ sử dụng format đơn giản tương tự LoRA cơ bản.

Tất cả các mẫu đều được chuẩn hóa theo template chat của Qwen, sử dụng thống nhất các token `<|im_start|>` và `<|im_end|>` nhằm đảm bảo tính nhất quán trong huấn luyện.

Cấu hình LoRA và huấn luyện

Cấu hình LoRA và các siêu tham số huấn luyện trong giai đoạn LoRA + RAFT được trình bày chi tiết trong Bảng 3.2.

Thành phần	Cấu hình
LoRA Dropout	$p = 0.1$, cao hơn so với LoRA cơ bản ($p = 0.05$) nhằm giảm nguy cơ overfitting khi xử lý ngữ cảnh dài và nhiều tài liệu tham khảo.
Batch size	$b = 2$ trên mỗi thiết bị GPU.
Gradient accumulation	$g = 16$, tương ứng với batch size hiệu dụng $B_{\text{eff}} = 32$.
Learning rate	$\eta = 2 \times 10^{-4}$.
Số epochs	$E = 2$.
Độ dài chuỗi tối đa	$L_{\text{max}} = 2048$ tokens, đủ để chứa câu hỏi và các tài liệu tham khảo trong RAFT.
Chiến lược đánh giá	steps , thực hiện đánh giá mỗi 500 bước huấn luyện để theo dõi quá trình hội tụ.
Gradient checkpointing	Bật, nhằm giảm mức sử dụng bộ nhớ GPU khi huấn luyện với chuỗi dài.
Mixed precision	bf16, giúp tiết kiệm bộ nhớ và tăng tốc độ huấn luyện.

Bảng 3.2: Cấu hình LoRA và huấn luyện trong giai đoạn LoRA + RAFT

Định dạng dữ liệu RAFT

Cấu trúc dữ liệu huấn luyện RAFT cho hai trường hợp có và không có context được trình bày trong Bảng 3.3.

Có context (Oracle / Distractor)	Không có context
<pre> < im_start >system Bạn là một AI tư vấn tuyển sinh đại học chuyên nghiệp. Sử dụng thông tin từ các tài liệu tham khảo để trả lời chính xác. < im_end > < im_start >user < context > ### Tài liệu chính (Oracle) [Nội dung oracle document] ### Tài liệu khác 1 [Nội dung distractor document 1] ### Tài liệu khác 2 [Nội dung distractor document 2] < im_end > Câu hỏi: ... < im_end > < im_start >assistant ### Tiêu đề... [Nội dung câu trả lời Markdown] < im_end > </pre>	<pre> < im_start >system Bạn là một AI tư vấn tuyển sinh đại học chuyên nghiệp. < im_end > < im_start >user Câu hỏi: ... < im_end > < im_start >assistant ### Tiêu đề... [Nội dung câu trả lời Markdown] < im_end > </pre>

Bảng 3.3: So sánh định dạng dữ liệu RAFT cho mẫu có context và không có context

Chiến lược tính loss và chia tập dữ liệu

Trong quá trình huấn luyện, loss chỉ được tính cho phần sinh câu trả lời (tokens nằm giữa `<|im_start|>assistant` và `<|im_end|>`), trong khi các token thuộc prompt (system, context và user) được gán nhãn `label = -100` để loại khỏi quá trình tối ưu.

3.3 Hệ thống tìm kiếm cục bộ (Local Search)

Hệ thống triển khai một kênh *Local Search* để truy vấn trên kho dữ liệu nội bộ đã được chuẩn hoá và phân đoạn sẵn (lưu trong file `local.pkl`). Nguồn dữ liệu này bao gồm các đoạn văn (chunks) gắn kèm siêu dữ liệu như mã trường (`school_id`) và khu vực nội dung (section) về quy chế, học phí, chương trình đào tạo, ... Mục tiêu là trả lời nhanh, ổn định cho các câu hỏi dựa trên thông tin chính thức, ít thay đổi, mà không phụ thuộc vào Web.

3.3.1 Kiến trúc và pipeline Local Search

Về logic, Local Search hoạt động theo chuỗi bước:

1. **Nạp và tổ chức dữ liệu nội bộ:** các tài liệu đã được xử lý offline được nạp từ

`local.pkl` thành tập các đoạn văn ngắn kèm siêu dữ liệu.

2. **Mã hoá ngữ nghĩa và lập chỉ mục:** mỗi đoạn văn được ánh xạ sang vector embedding bằng một mô hình ngữ nghĩa đa ngôn ngữ; toàn bộ tập vector được lập chỉ mục trong một cơ sở dữ liệu vector (FAISS) để hỗ trợ tìm kiếm nhanh.
3. **Sinh truy vấn nội bộ từ Router:** mô hình Router (Qwen3-4B LoRA) phân tích câu hỏi gốc và sinh ra danh sách truy vấn có cấu trúc, trong đó có `query`, `school_id`, `section`,... dùng làm tín hiệu để giới hạn phạm vi tìm kiếm.
4. **Tìm kiếm ngữ nghĩa có lọc:** với mỗi truy vấn, hệ thống: (i) lọc trước các đoạn văn theo trường/nội dung (nếu được chỉ định), sau đó (ii) tìm kiếm theo độ tương đồng ngữ nghĩa trong không gian embedding để lấy ra một số ít đoạn văn liên quan nhất.
5. **Chuẩn hoá kết quả cho RAG:** các đoạn văn được gộp và chuẩn hoá thành danh sách *nguồn* (chunks) dùng làm ngữ cảnh đầu vào cho bước RAG của mô hình sinh (Qwen3-4B LoRA).

3.3.2 Phối hợp Local Search với Web Search

Local Search không hoạt động tách biệt mà được điều phối bởi lớp Router chung với Web Search. Khi người dùng bật cả hai kênh, hệ thống thực hiện:

1. **Ưu tiên Local Search:** Router trước hết sinh truy vấn nội bộ và gọi Local Search để thu thập các đoạn văn liên quan.
2. **Đánh giá chất lượng kết quả cục bộ:** các chunk từ Local Search được chấm điểm lại bằng mô hình Cross-Encoder chuyên cho reranking (so sánh trực tiếp câu hỏi–đoạn văn) và lọc theo một ngưỡng tối thiểu.
3. **Quyết định sử dụng kênh nào:** nếu số lượng/độ phủ chunk nội bộ đạt tiêu chí, hệ thống chỉ dùng ngữ cảnh từ Local Search cho bước RAG; ngược lại, Router tự động *fallback* sang pipeline Web Search.

Nhờ cơ chế này, Local Search được dùng cho các truy vấn về quy định và thông tin nội bộ có sẵn trong kho dữ liệu tĩnh, trong khi Web Search đảm trách các câu hỏi mang tính thời sự hoặc ngoài phạm vi `local.pkl`.

3.4 Tìm kiếm thông tin từ Web (Web Search)

Để khắc phục hạn chế về dữ liệu tĩnh của mô hình ngôn ngữ và đảm bảo khả năng trả lời các câu hỏi mang tính thời sự (như tin tức tuyển sinh mới, thay đổi giờ chót về đề án hay điểm chuẩn năm 2025), nhóm nghiên cứu đã xây dựng một phân hệ tìm kiếm

Web (Web Search Pipeline) chuyên biệt. Quy trình xử lý được thiết kế kết hợp giữa **cơ chế xử lý song song (Parallel Processing)** để tối ưu tốc độ và **cơ chế lọc đa tầng (Multi-stage Filtering)** để đảm bảo thông tin đầu vào là sạch và xác thực nhất.



Hình 3.2: Kiến trúc hệ thống WebSearch

3.4.1 Tổng quan Kiến trúc Web Search

Kiến trúc hệ thống Web Search hoạt động theo luồng dữ liệu gồm 5 giai đoạn chính. Điểm đặc biệt của kiến trúc này là khả năng phân rã một câu hỏi thành nhiều truy vấn con và xử lý chúng đồng thời trên các luồng riêng biệt:

1. **Sinh truy vấn (Query Generation):** Biến đổi câu hỏi người dùng thành các từ khóa tìm kiếm tối ưu.
2. **Truy tìm và Xếp hạng trang Song song (Parallel Search & Page Reranking):** Tìm kiếm và lọc URL bằng mô hình ngôn ngữ nhỏ (SLM) trên đa luồng.
3. **Thu thập dữ liệu (Crawling & Parsing):** Tải và chuẩn hóa nội dung từ đa dạng định dạng (Text, PDF, Image).
4. **Xử lý phân đoạn và Lọc chuyên sâu (Chunking, Deduplication & Semantic Filtering):** Chia nhỏ văn bản, khử trùng lặp và lọc ngữ nghĩa bằng Cross-Encoder.
5. **Tích hợp RAG:** Hợp nhất kết quả và tổng hợp ngữ cảnh đưa vào mô hình sinh câu trả lời.

3.4.2 Phân tích và Sinh truy vấn (Query Generation)

Người dùng thường có xu hướng đặt câu hỏi ngắn gọn, thiếu ngữ cảnh (ví dụ: *"Học phí thế nào?"*). Nếu đưa trực tiếp câu hỏi này vào công cụ tìm kiếm, kết quả trả về thường bị nhiễu.

Hệ thống sử dụng mô hình ngôn ngữ chính (**Qwen-4B LoRA + RAFT**) để thực hiện kỹ thuật **Query Expansion (Mở rộng truy vấn)**. Mô hình phân tích ý định người dùng và sinh ra N truy vấn tìm kiếm (search queries) cụ thể hơn (thường từ 2-3 truy vấn), chứa các từ khóa trọng tâm như tên trường, năm tuyển sinh 2025, mã ngành.

Ngay sau khi các truy vấn được sinh ra, hệ thống kích hoạt cơ chế **xử lý song song**, khởi tạo các luồng độc lập để gửi yêu cầu đến API tìm kiếm (Google Custom Search và Brave Search), đảm bảo giảm thiểu độ trễ chờ đợi.

3.4.3 Truy xuất và Xếp hạng trang (Search & Page Reranking)

Kết quả trả về từ API trên mỗi luồng thường bao gồm cả những trang quảng cáo hoặc bài viết SEO kém chất lượng. Để tối ưu hóa tài nguyên tính toán cho bước Crawl (vốn tốn kém thời gian), hệ thống áp dụng một bước lọc sơ cấp ngay trên từng luồng.

Nhóm sử dụng mô hình ngôn ngữ siêu nhỏ **BGE-M3**. (đã được tinh chỉnh nhẹ cho tác vụ phân loại) đóng vai trò là bộ lọc (Page Reranker). Mô hình này đọc Tiêu đề (Title) và Đoạn trích dẫn (Snippet) của kết quả tìm kiếm, sau đó chấm điểm độ liên quan so với câu hỏi gốc. Chỉ những URL đạt ngưỡng điểm tin cậy nhất định mới được chuyển sang giai đoạn thu thập dữ liệu. Việc sử dụng model giúp bước lọc này diễn ra gần như tức thời.

3.4.4 Thu thập và Tiền xử lý dữ liệu (Crawling & Parsing)

Tại bước này, các luồng tiếp tục truy cập vào các URL đã được chọn lọc để tải nội dung chi tiết. Bộ công cụ Crawl được thiết kế để xử lý đa dạng định dạng dữ liệu:

- **Văn bản (HTML/Text):** Loại bỏ các thành phần giao diện (menu, footer, quảng cáo) để lấy nội dung chính.
- **Tài liệu PDF:** Sử dụng thư viện chuyên dụng để trích xuất văn bản từ các file pdf.
- **Hình ảnh:** Sử dụng kỹ thuật OCR để trích xuất thông tin từ các ảnh chụp thông báo, bảng điểm chuẩn dạng ảnh.

Dữ liệu sau khi thu thập sẽ được chuẩn hóa về định dạng văn bản thô (plain text) hoặc Markdown.

3.4.5 Phân đoạn, Khử trùng lặp và Lọc ngữ nghĩa (Chunking, Deduplication & Semantic Filtering)

Đây là giai đoạn quan trọng nhất để tinh lọc thông tin ("Needle in a haystack"), diễn ra song song trên từng luồng dữ liệu:

Bước 1: Phân đoạn (Chunking). Văn bản được chia thành các đoạn nhỏ (chunks) kích thước cố định (ví dụ: 512 tokens) có độ trượt (overlap) để giữ ngữ cảnh.

Bước 2: Khử trùng lặp (Deduplication). Do thông tin tuyển sinh thường được đăng tải lại trên nhiều trang báo, hệ thống sử dụng thuật toán Cosine Similarity để loại bỏ các đoạn văn bản có nội dung trùng lặp hoặc gần giống nhau, giúp giảm nhiễu.

Bước 3: Lọc ngữ nghĩa (Semantic Filtering). Các chunk còn lại được đưa qua mô hình Cross-Encoder BGE-M3. Khác với việc sắp xếp thông thường, ở đây mô hình đóng vai trò là một **bộ lọc ngưỡng (Gatekeeper)**. Nó so sánh trực tiếp ý nghĩa

của câu hỏi và đoạn văn bản. Chỉ những đoạn văn có điểm tương đồng ngữ nghĩa vượt qua ngưỡng thiết lập (θ) mới được giữ lại; các đoạn văn không liên quan sẽ bị loại bỏ hoàn toàn.

3.4.6 Hợp nhất ngữ cảnh và Tạo câu trả lời (Final RAG)

Sau khi các luồng song song hoàn tất quá trình lọc, hệ thống thực hiện hợp nhất (merge) các đoạn văn bản sạch (Unique Chunks) từ tất cả các truy vấn lại với nhau.

Danh sách các đoạn văn bản giá trị nhất này được đưa vào Prompt (câu lệnh) khuôn mẫu bao gồm: Câu hỏi gốc + Ngữ cảnh đã lọc + Các chỉ dẫn ràng buộc (như "trích dẫn nguồn"). Toàn bộ nội dung được đưa vào mô hình ngôn ngữ chính (SLM) để sinh ra câu trả lời cuối cùng, đảm bảo tính chính xác, ngắn gọn và có cơ sở dữ liệu xác thực từ Web.

Chương 4

KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Phương pháp Đánh giá

Để đánh giá chất lượng đầu ra của hệ thống chatbot tư vấn tuyển sinh, nhóm nghiên cứu sử dụng kết hợp hai phương pháp đánh giá: **Human Evaluation** và **LLM-as-a-Judge**. Hai phương pháp này cho phép phản ánh sát chất lượng thực tế của câu trả lời trong bối cảnh hội thoại và tư vấn giáo dục.

Human Evaluation

Đánh giá bởi con người được thực hiện nhằm đo lường mức độ hữu ích và độ tin cậy của câu trả lời từ góc nhìn người dùng cuối. Các câu trả lời của chatbot được chấm điểm dựa trên các tiêu chí chính gồm: tính chính xác của thông tin, mức độ đầy đủ và sự phù hợp với ngữ cảnh tuyển sinh. Kết quả đánh giá được tổng hợp theo thang điểm Likert từ 1 đến 5.

LLM-as-a-Judge

Song song với đánh giá thủ công, nhóm nghiên cứu sử dụng phương pháp *LLM-as-a-Judge*, trong đó một mô hình ngôn ngữ lớn đóng vai trò giám khảo để đánh giá và so sánh chất lượng các câu trả lời do hệ thống sinh ra. Phương pháp này cho phép đánh giá nhất quán trên tập dữ liệu lớn, đồng thời hỗ trợ so sánh hiệu quả giữa các phiên bản mô hình sau các giai đoạn huấn luyện khác nhau.

4.2 Kết quả Đánh giá Huấn luyện

4.2.1 Kết quả Huấn luyện Định lượng

Sau khi hoàn thành quá trình training, kết quả thu được như sau:

- **Giai đoạn 1 (LoRA cơ bản):** Train loss giảm từ ~ 2.5 xuống ~ 0.8 . Eval loss giảm xuống ~ 1.0 . Thời gian training khoảng 2–3 giờ.

- **Giai đoạn 2 (LoRA + RAFT):** Train loss giảm từ ~ 2.5 xuống ~ 1.2 (cao hơn giai đoạn 1 do bài toán phức tạp hơn). Eval loss giảm xuống ~ 1.3 . Thời gian training khoảng 4–5 giờ.

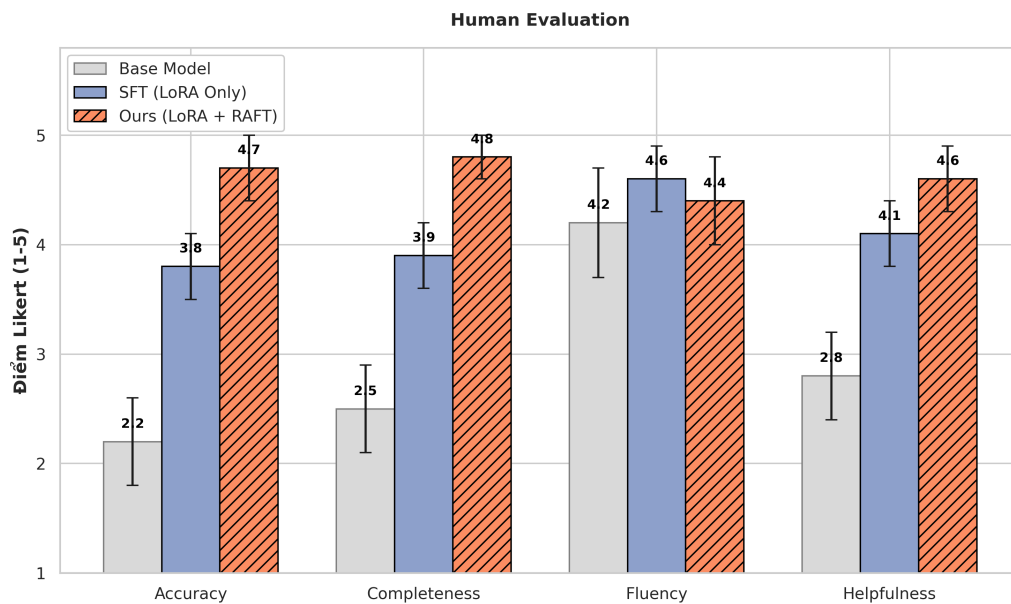
Bảng dưới đây so sánh chi tiết các metrics giữa hai giai đoạn:

Metric	LoRA cơ bản	LoRA + RAFT
Train loss (final)	~ 0.8	~ 1.2
Eval loss (final)	~ 1.0	~ 1.3
Training time	$\sim 2\text{--}3$ giờ	$\sim 4\text{--}5$ giờ
Memory usage	Thấp	Trung bình
Max sequence length	512 tokens	2048 tokens
Effective batch size	8	32

Bảng 4.1: So sánh metrics huấn luyện giữa LoRA cơ bản và LoRA + RAFT

4.3 Kết quả Đánh giá hệ thống

Kết quả thực nghiệm từ cả đánh giá thủ công (Human Eval) và tự động (LLM-as-a-Judge) cho thấy sự cải thiện rõ rệt qua từng giai đoạn huấn luyện, đồng thời cũng bộc lộ những đặc tính thú vị về sự đánh đổi (trade-off) giữa các kỹ thuật. Nhận xét về Đánh

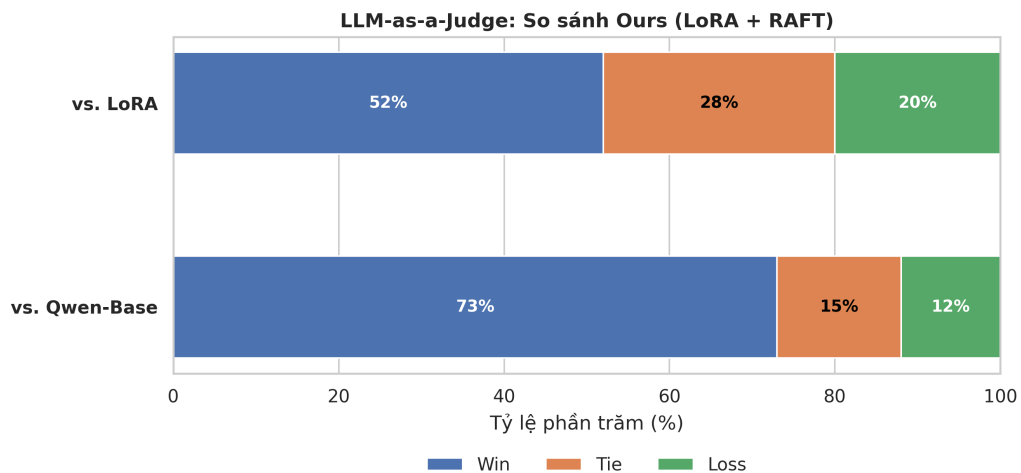


Hình 4.1: Human Evaluation

giá Con người (Hình 4.2.1):

- **Tính chính xác (Accuracy):** Mô hình **LoRA+RAFT** đạt điểm số vượt trội (4.7/5.0) so với LoRA cơ bản (3.8/5.0). Điều này khẳng định hiệu quả của kỹ thuật RAFT trong việc ép buộc mô hình trích xuất dữ liệu thực tế từ tài liệu tham khảo thay vì dựa vào trí nhớ nội tại dễ gây ảo giác.

- **Độ trôi chảy (Fluency):** Một quan sát đáng chú ý là phiên bản **LoRA cơ bản (SFT)** lại đạt điểm trôi chảy cao hơn (4.6) so với phiên bản RAFT (4.4). Nguyên nhân là do mô hình SFT được tự do sinh văn bản dựa trên xác suất ngôn ngữ tự nhiên, trong khi mô hình RAFT bị ràng buộc bởi nội dung của các đoạn văn bản trích xuất (vốn thường là văn bản hành chính khô khan hoặc rời rạc). Tuy nhiên, mức giảm nhẹ này là hoàn toàn chấp nhận được để đổi lấy độ chính xác cao – yếu tố sống còn của bài toán tuyển sinh.



Hình 4.2: LLM-as-a-Judge

Nhận xét về LLM-as-a-Judge (Hình 4.2.2):

- **So với Qwen-Base:** Hệ thống đề xuất chiến thắng áp đảo (73%), chứng minh sự cần thiết của việc tinh chỉnh mô hình cho tiếng Việt chuyên ngành.
- **So với LoRA:** Mặc dù phiên bản RAFT chiến thắng 52% nhờ khả năng trả lời đúng các câu hỏi về số liệu (điểm chuẩn, học phí), nhưng vẫn có tỷ lệ **thua 20%** (Loss Rate). Phân tích kỹ các trường hợp thua cho thấy, mô hình RAFT thường gặp khó khăn trong các câu hỏi giao tiếp xã giao (chit-chat) hoặc các câu hỏi quá chung chung không tìm thấy tài liệu tham khảo, dẫn đến việc từ chối trả lời hoặc trả lời cụt lủn, trong khi LoRA SFT vẫn phản hồi mượt mà.

Chương 5

KẾT LUẬN

5.1 Tổng kết Hệ thống

Đã hoàn thiện phiên bản cải tiến với model fine-tune, router web/local tốt hơn, vectordb làm sạch và UX đầy đủ (nguồn, lịch sử, đánh giá).

5.2 Tính năng Chính

- Qwen3-4B (LoRA + RAFT) cho miền tuyển sinh.
- Router thông minh kết hợp Web Search + Local RAG.
- Vector database 200+ trường, 4 chuyên mục, cập nhật định kỳ.
- vLLM inference Kaggle/Local, streaming response.

5.3 Hướng Phát triển Tương lai

- Bổ sung kiểm chứng nguồn (fact-check) và đánh giá tự động sau suy luận.
- Mở rộng dữ liệu sang chương trình liên kết, sau đại học.
- Tối ưu chi phí hạ tầng, thử nghiệm mô hình 8B/14B với quantization.
- Bổ sung dashboard giám sát chất lượng và tự động làm mới dữ liệu.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, Advances in Neural Information Processing Systems (NeurIPS), 2020. Available: <https://arxiv.org/abs/2005.11401>
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, *LoRA: Low-Rank Adaptation of Large Language Models*, Proceedings of the International Conference on Learning Representations (ICLR), 2022. arXiv:2106.09685.
- [3] Y. Zhang, Y. Sun, H. Xu, Z. Liu, and M. Zhou, *RAFT: Retrieval-Augmented Fine-Tuning for Language Models*, arXiv preprint arXiv:2304.06767, 2023.
- [4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, M. Wainwright, et al., *Training Language Models to Follow Instructions with Human Feedback*, Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [5] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, arXiv preprint arXiv:2305.18290, 2023.
- [6] J. Chen, S. Xiao, P. Zhou, K. Wang, J. Liu, and Z. Zhang, *BGE-M3: Multi-Lingual, Multi-Granular, Multi-Functional Text Embeddings*, arXiv preprint arXiv:2402.03216, 2024.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, and I. Stoica, *vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention*, arXiv preprint arXiv:2309.06180, 2023.